H.T.NO:2303A51868

BATCH 28 TASK

1:

Prompt:

Use AI to generate a Python script for cleaning a customer dataset that handle missing values in columns (age, income, city),Remove duplicate records, Standardize text values (e.g., city names in lowercase) and Encode categorical variables (gender, city).

Data Cleaning Steps

I will perform the following steps to clean the shopping_trends.csv dataset:

1. **Handle Missing Values**: Impute missing values in 'age' and 'income' with their respective means, and in 'city' with its mode.
2. **Remove Duplicate Records**: Eliminate any duplicate rows in the dataset.
3. **Standardize Text Values**: Convert 'city' names to lowercase for consistency.
4. **Encode Categorical Variables**: Encode 'gender' to numerical values (e.g., 0/1) and 'city' using one-hot encoding.

```
import pandas as pd

# Load the dataset
try:
    df = pd.read_csv("shopping_trends.csv")
    print("Original DataFrame head:")
    display(df.head())
    print("\nOriginal DataFrame info:")
    df.info()
except FileNotFoundError:
    print("Error: shopping_trends.csv not found. Please upload the file or provide the correct path.")
    df = pd.DataFrame() # Create an empty DataFrame to avoid further errors
```

```
Original DataFrame head:
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

```
Original DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Customer ID               3900 non-null   int64
 1   Age                       3900 non-null   int64
 2   Gender                    3900 non-null   object
 3   Item Purchased            3900 non-null   object
 4   Category                  3900 non-null   object
 5   Purchase Amount (USD)     3900 non-null   int64
 6   Location                  3900 non-null   object
 7   Size                      3900 non-null   object
 8   Color                     3900 non-null   object
 9   Season                    3900 non-null   object
 10  Review Rating             3900 non-null   float64
 11  Subscription Status       3900 non-null   object
 12  Payment Method            3900 non-null   object
 13  Shipping Type             3900 non-null   object
 14  Discount Applied          3900 non-null   object
```

```
 15  Promo Code Used           3900 non-null   object
 16  Previous Purchases        3900 non-null   int64
 17  Preferred Payment Method  3900 non-null   object
 18  Frequency of Purchases    3900 non-null   object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB
```

1. Handle Missing Values

- **Numerical columns (age, income):** Impute with the mean.
- **Categorical column (city):** Impute with the mode.

```python
if not df.empty:
    # Impute 'age' with mean
    if 'age' in df.columns:
        df['age'].fillna(df['age'].mean(), inplace=True)

    # Impute 'income' with mean
    if 'income' in df.columns:
        df['income'].fillna(df['income'].mean(), inplace=True)

    # Impute 'city' with mode
    if 'city' in df.columns:
        mode_city = df['city'].mode()[0]
        df['city'].fillna(mode_city, inplace=True)

    print("DataFrame after handling missing values:")
    df.info()
```

```
DataFrame after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Customer ID              3900 non-null   int64
 1   Age                      3900 non-null   int64
 2   Gender                   3900 non-null   object
 3   Item Purchased           3900 non-null   object
 4   Category                 3900 non-null   object
 5   Purchase Amount (USD)    3900 non-null   int64
 6   Location                 3900 non-null   object
 7   Size                     3900 non-null   object
 8   Color                    3900 non-null   object
 9   Season                   3900 non-null   object
 10  Review Rating            3900 non-null   float64
 11  Subscription Status      3900 non-null   object
 12  Payment Method           3900 non-null   object
 13  Shipping Type            3900 non-null   object
 14  Discount Applied         3900 non-null   object
 15  Promo Code Used          3900 non-null   object
 16  Previous Purchases       3900 non-null   int64
 17  Preferred Payment Method 3900 non-null   object
 18  Frequency of Purchases   3900 non-null   object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB
```

2. Remove Duplicate Records

```python
if not df.empty:
    initial_rows = len(df)
    df.drop_duplicates(inplace=True)
    print(f"Number of duplicate records removed: {initial_rows - len(df)}")
    print("DataFrame after removing duplicates:")
    df.info()
```

```
Number of duplicate records removed: 0
DataFrame after removing duplicates:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Customer ID               3900 non-null   int64
 1   Age                       3900 non-null   int64
 2   Gender                    3900 non-null   object
 3   Item Purchased            3900 non-null   object
 4   Category                  3900 non-null   object
 5   Purchase Amount (USD)     3900 non-null   int64
 6   Location                  3900 non-null   object
 7   Size                      3900 non-null   object
 8   Color                     3900 non-null   object
 9   Season                    3900 non-null   object
 10  Review Rating             3900 non-null   float64
 11  Subscription Status       3900 non-null   object
 12  Payment Method            3900 non-null   object
 13  Shipping Type             3900 non-null   object
 14  Discount Applied          3900 non-null   object
 15  Promo Code Used           3900 non-null   object
 16  Previous Purchases        3900 non-null   int64
 17  Preferred Payment Method  3900 non-null   object
 18  Frequency of Purchases    3900 non-null   object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB
```

3. Standardize Text Values (City Names to Lowercase)

```
[25]
✓ 0s
if not df.empty and 'city' in df.columns:
    df['city'] = df['city'].str.lower()
    print("Unique cities after standardization:")
    display(df['city'].unique())
```

4. Encode Categorical Variables

- **Gender**: Using a simple mapping (e.g., 'Male' -> 0, 'Female' -> 1).
- **City**: Using one-hot encoding.

```python
if not df.empty:
    # Encode 'gender'
    if 'gender' in df.columns:
        gender_mapping = {'Male': 0, 'Female': 1}
        df['gender_encoded'] = df['gender'].map(gender_mapping)
        print("\n'gender' column after encoding:")
        display(df[['gender', 'gender_encoded']].head())

    # One-hot encode 'city'
    if 'city' in df.columns:
        df = pd.get_dummies(df, columns=['city'], prefix='city', dtype=int)
        print("\nDataFrame after one-hot encoding 'city' (first 5 rows with new columns):")
        display(df.filter(like='city_').head())

    print("\nCleaned DataFrame head:")
    display(df.head())
    print("\nCleaned DataFrame info:")
    df.info()
```

Cleaned DataFrame head:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

Cleaned DataFrame info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3900 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Payment Method          3900 non-null   object
 13  Shipping Type           3900 non-null   object
 14  Discount Applied        3900 non-null   object
 15  Promo Code Used         3900 non-null   object
 16  Previous Purchases      3900 non-null   int64
 17  Preferred Payment Method  3900 non-null   object
 18  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB
```

TASK 2:

Prompt:

Preprocess sales transaction data using AI assistance that converts date columns into datetime format, Create new features (month, year, day of week),Normalize sales amount column, and Detect and handle outliers in dataset.

```python
import pandas as pd

# Load the sales data from the CSV file
df = pd.read_csv('sales_data.csv')

# Display the first 5 rows to understand the data structure
print("First 5 rows of the dataset:")
display(df.head())

# Display information about the DataFrame, including data types and non-null counts
print("\nDataFrame Info:")
display(df.info())
```

```
First 5 rows of the dataset:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Product_ID           1000 non-null   int64
 1   Sale_Date            1000 non-null   object
 2   Sales_Rep            1000 non-null   object
 3   Region               1000 non-null   object
 4   Sales_Amount         1000 non-null   float64
 5   Quantity_Sold        1000 non-null   int64
 6   Product_Category     1000 non-null   object
 7   Unit_Cost            1000 non-null   float64
 8   Unit_Price           1000 non-null   float64
 9   Customer_Type        1000 non-null   object
 10  Discount             1000 non-null   float64
 11  Payment_Method       1000 non-null   object
 12  Sales_Channel        1000 non-null   object
 13  Region_and_Sales_Rep 1000 non-null   object
dtypes: float64(4), int64(2), object(8)
memory usage: 109.5+ KB
None
```

```python
# Convert 'Sale_Date' to datetime format
df['Sale_Date'] = pd.to_datetime(df['Sale_Date'])

# Create new features: Month, Year, Day of Week
df['Month'] = df['Sale_Date'].dt.month
df['Year'] = df['Sale_Date'].dt.year
df['Day_of_Week'] = df['Sale_Date'].dt.day_name()

print("DataFrame after adding date features:")
display(df.head())
```

```
DataFrame after adding date features:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	Month	Year	Day_of_Week
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2	2023	Friday
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	4	2023	Friday
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	9	2023	Thursday
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	8	2023	Thursday
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	3	2023	Friday

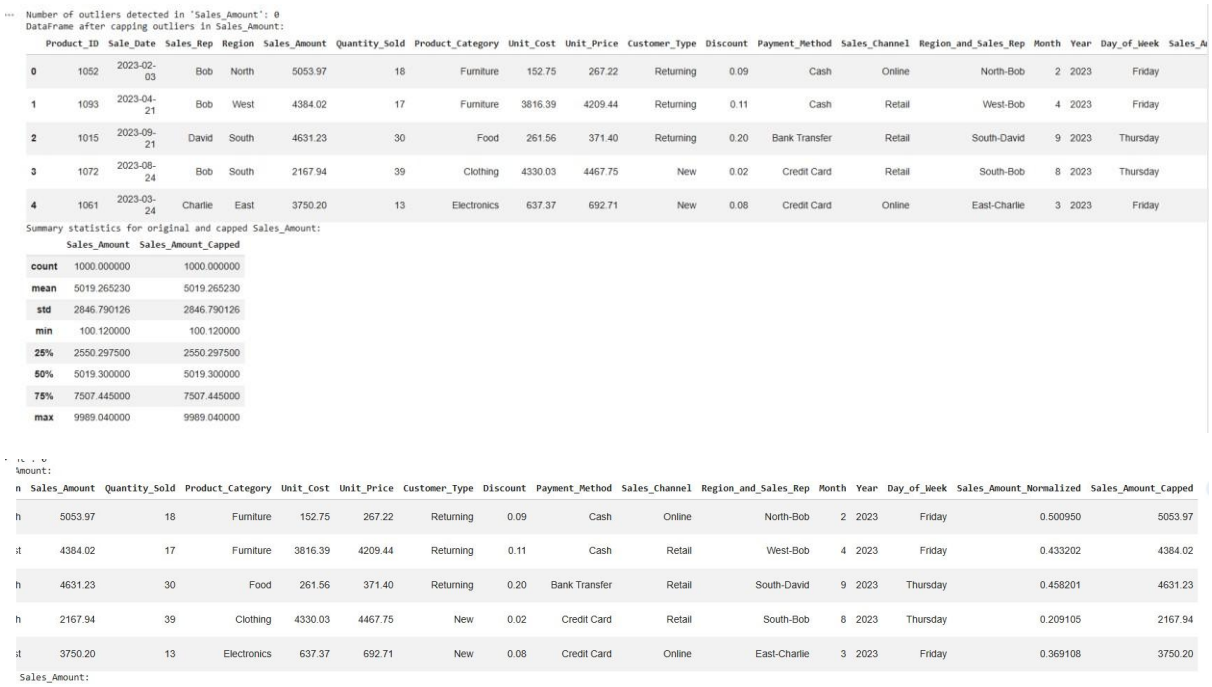
```python
from sklearn.preprocessing import MinMaxScaler

# Normalize 'Sales_Amount' column
scaler = MinMaxScaler()
df['Sales_Amount_Normalized'] = scaler.fit_transform(df[['Sales_Amount']])

print("DataFrame after normalizing Sales_Amount:")
display(df.head())
```

```
DataFrame after normalizing Sales_Amount:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	Month	Year	Day_of_Week	Sales_A
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2	2023	Friday	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	4	2023	Friday	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	9	2023	Thursday	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	8	2023	Thursday	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	3	2023	Friday	

n	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	Month	Year	Day_of_Week	Sales_Amount_Normalized	Sales_Amount_Capped
h	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2	2023	Friday	0.500950	5053.97
t	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	4	2023	Friday	0.433202	4384.02
h	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	9	2023	Thursday	0.458201	4631.23
h	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	8	2023	Thursday	0.209105	2167.94
t	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	3	2023	Friday	0.369108	3750.20

```python
# Detect and handle outliers in 'Sales_Amount' using IQR method
Q1 = df['Sales_Amount'].quantile(0.25)
Q3 = df['Sales_Amount'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers_df = df[(df['Sales_Amount'] < lower_bound) | (df['Sales_Amount'] > upper_bound)]

print(f"Number of outliers detected in 'Sales_Amount': {len(outliers_df)}")

# Handle outliers (e.g., capping them)
df['Sales_Amount_Capped'] = df['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)

print("DataFrame after capping outliers in Sales_Amount:")
display(df.head())

print("Summary statistics for original and capped Sales_Amount:")
display(df[['Sales_Amount', 'Sales_Amount_Capped']].describe())
```



```
Number of outliers detected in 'Sales_Amount': 0
DataFrame after capping outliers in Sales_Amount:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	Month	Year	Day_of_Week	Sales_A
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2	2023	Friday	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	4	2023	Friday	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	9	2023	Thursday	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	8	2023	Thursday	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	3	2023	Friday	

```
Summary statistics for original and capped Sales_Amount:
```

	Sales_Amount	Sales_Amount_Capped
count	1000.000000	1000.000000
mean	5019.265230	5019.265230
std	2846.790126	2846.790126
min	100.120000	100.120000
25%	2550.297500	2550.297500
50%	5019.300000	5019.300000
75%	7507.445000	7507.445000
max	9989.040000	9989.040000

```
mount:
```

n	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	Month	Year	Day_of_Week	Sales_Amount_Normalized	Sales_Amount_Capped
h	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob	2	2023	Friday	0.500950	5053.97
t	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	4	2023	Friday	0.433202	4384.02
h	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	9	2023	Thursday	0.458201	4631.23
h	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob	8	2023	Thursday	0.209105	2167.94
t	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	3	2023	Friday	0.369108	3750.20

```
Sales_Amount:
```

TASK 3:

Prompt:

Clean and preprocess a healthcare dataset using AI Scripts that handle missing values in blood_presure, cholesterol, Encode Categorical Features, Scale numerical features(min-max or standard scaling), and split the dataset into training and testing datasets.

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Define features (these variables are already present in the kernel from previous execution)
# numerical_features = ['age', 'blood_pressure', 'cholesterol']
# categorical_features = ['gender', 'medical_condition']
# target_feature = 'heart_disease'

# Create preprocessing pipelines for numerical and categorical features
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),  # Handle missing values with mean imputation
    ('scaler', StandardScaler())                  # Scale numerical features
])

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # One-hot encode categorical features
])

# Combine preprocessing steps using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Apply preprocessing to the features (X) and separate the target (y)
X = df.drop(columns=[target_feature])
y = df[target_feature]

X_preprocessed = preprocessor.fit_transform(X)
```

```python
# Get feature names after one-hot encoding
categorical_feature_names = preprocessor.named_transformers_['cat']['onehot'].get_feature_names_out(categorical_features)
all_feature_names = numerical_features + list(categorical_feature_names)

X_preprocessed_df = pd.DataFrame(X_preprocessed, columns=all_feature_names)

print("\nPreprocessed Dataset Head:")
display(X_preprocessed_df.head())
print("\nShape of preprocessed dataset:", X_preprocessed_df.shape)
```

```
Preprocessed Dataset Head:
```

	age	blood_pressure	cholesterol	gender_Female	gender_Male	medical_condition_Asthma	medical_condition_Diabetes	medical_condition_Hypertension	medical_condition_None
0	0.500094	-1.740043e-01	8.986465e-16	0.0	1.0	0.0	0.0	1.0	0.0
1	1.249514	-2.021231e+00	-1.372887e+00	0.0	1.0	0.0	0.0	0.0	1.0
2	-0.076383	1.287666e-01	-8.293415e-01	0.0	1.0	0.0	1.0	0.0	0.0
3	-0.883451	-4.277508e-02	-6.890447e-02	1.0	0.0	1.0	0.0	0.0	0.0
4	0.730685	-9.768167e-16	-2.458388e+00	1.0	0.0	0.0	0.0	1.0	0.0

```
Shape of preprocessed dataset: (200, 9)
```

```
# Split the preprocessed data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_preprocessed_df, y, test_size=0.2, random_state=42, stratify=y)

print("\nShape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

print("\nPreprocessing complete. The dataset is ready for machine learning models.")
```

```
Shape of X_train: (160, 9)
Shape of X_test: (40, 9)
Shape of y_train: (160,)
Shape of y_test: (40,)

Preprocessing complete. The dataset is ready for machine learning models.
```

TASK 4:

Prompt:

Clean and preprocess an employee salary dataset using AI Scripts that handle missing values, Detect and remove salary outliers, Standardize department names(lowercase), Apply label encoding to job location, department.

Data Cleaning and Preprocessing

First, let's create a sample employee salary dataset for demonstration purposes.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Create a sample DataFrame for demonstration
data = {
    'employee_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    'department': [
        'Sales', 'Marketing', 'IT', 'HR', 'Sales', 'Marketing', 'IT', 'HR',
        'sales', 'marketing', 'it', 'hr', 'Sales', 'Marketing', 'IT'
    ],
    'job_location': [
        'New York', 'London', 'Paris', 'New York', 'London', 'Paris', 'New York',
        'London', 'new york', 'london', 'paris', 'new york', 'London', 'Paris', 'New York'
    ],
    'salary': [
        60000, 75000, 80000, 55000, 62000, 78000, 85000, 58000,
        61000, 76000, 90000, 57000, 150000, 20000, 700000 # Introduce some outliers
    ],
    'years_experience': [
        2, 5, 3, 1, 2, 4, 6, 2,
        3, 5, None, 1, 7, 1, 8
    ]
}

df = pd.DataFrame(data)
print("Original DataFrame head:")
display(df.head())
print("\nOriginal DataFrame info:")
df.info()
```

```
Original DataFrame head:
```

	employee_id	department	job_location	salary	years_experience
0	1	Sales	New York	60000	2.0
1	2	Marketing	London	75000	5.0
2	3	IT	Paris	80000	3.0
3	4	HR	New York	55000	1.0
4	5	Sales	London	62000	2.0

```
Original DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   employee_id       15 non-null     int64
 1   department        15 non-null     object
 2   job_location      15 non-null     object
 3   salary            15 non-null     int64
 4   years_experience  14 non-null     float64
dtypes: float64(1), int64(2), object(2)
memory usage: 732.0+ bytes
```

1. Handle Missing Values

We'll check for missing values and fill years_experience with the median, and drop rows with any other missing values. You can adjust the strategy (e.g., fill with mean, mode, or use imputation techniques) based on your dataset's characteristics.

```
print("Missing values before handling:")
display(df.isnull().sum())

# Fill 'years_experience' with its median
if 'years_experience' in df.columns and df['years_experience'].isnull().any():
    median_experience = df['years_experience'].median()
    df['years_experience'] = df['years_experience'].fillna(median_experience)
    print(f"\nFilled missing 'years_experience' with median: {median_experience}")

# Drop rows with any remaining missing values
df.dropna(inplace=True)
print("\nMissing values after handling:")
display(df.isnull().sum())

print("\nDataFrame head after handling missing values:")
display(df.head())
```

```
Missing values before handling:
```

	0
employee_id	0
department	0
job_location	0
salary	0
years_experience	1

dtype: int64

```
Filled missing 'years_experience' with median: 3.0

Missing values after handling:
```

	0
employee_id	0
department	0
job_location	0
salary	0
years_experience	0

dtype: int64

```
DataFrame head after handling missing values:
```

	employee_id	department	job_location	salary	years_experience
0	1	Sales	New York	60000	2.0
1	2	Marketing	London	75000	5.0
2	3	IT	Paris	80000	3.0
3	4	HR	New York	55000	1.0
4	5	Sales	London	62000	2.0

2. Detect and Remove Salary Outliers

We'll use the Interquartile Range (IQR) method to detect and remove outliers from the 'salary' column. This method defines outliers as values that fall below Q1 - 1.5 * IQR or above Q3 + 1.5 * IQR.

```python
if 'salary' in df.columns:
    Q1 = df['salary'].quantile(0.25)
    Q3 = df['salary'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df['salary'] < lower_bound) | (df['salary'] > upper_bound)]
    df_cleaned = df[~((df['salary'] < lower_bound) | (df['salary'] > upper_bound))]

    print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
    print(f"Lower Bound: {lower_bound}, Upper Bound: {upper_bound}")
    print(f"\nNumber of salary outliers detected: {len(outliers)}")
    print("Salary Outliers:")
    display(outliers)

    df = df_cleaned.reset_index(drop=True)
    print("\nDataFrame head after removing salary outliers:")
    display(df.head())
else:
    print("Salary column not found in the DataFrame.")
```

```
Q1: 59000.0, Q3: 82500.0, IQR: 23500.0
Lower Bound: 23750.0, Upper Bound: 117750.0

Number of salary outliers detected: 3
Salary Outliers:
```

	employee_id	department	job_location	salary	years_experience
12	13	Sales	London	150000	7.0
13	14	Marketing	Paris	20000	1.0
14	15	IT	New York	700000	8.0

```
DataFrame head after removing salary outliers:
```

	employee_id	department	job_location	salary	years_experience
0	1	Sales	New York	60000	2.0
1	2	Marketing	London	75000	5.0
2	3	IT	Paris	80000	3.0
3	4	HR	New York	55000	1.0
4	5	Sales	London	62000	2.0

3. Standardize Department Names (lowercase)

This step ensures consistency across department names by converting all entries to lowercase.

```
if 'department' in df.columns:
    df['department'] = df['department'].str.lower()
    print("DataFrame head after standardizing department names:")
    display(df.head())
else:
    print("Department column not found in the DataFrame.")
```

```
DataFrame head after standardizing department names:
```

	employee_id	department	job_location	salary	years_experience
0	1	sales	New York	60000	2.0
1	2	marketing	London	75000	5.0
2	3	it	Paris	80000	3.0
3	4	hr	New York	55000	1.0
4	5	sales	London	62000	2.0

4. Apply Label Encoding to Job Location and Department

Label Encoding converts categorical text data into numerical labels, which is necessary for many machine learning models. Each unique category will be assigned a unique integer.

```
le = LabelEncoder()

if 'job_location' in df.columns:
    df['job_location_encoded'] = le.fit_transform(df['job_location'])
    print("Job Location Mappings:")
    for i, item in enumerate(le.classes_):
        print(f"{item}: {i}")
else:
    print("Job location column not found in the DataFrame.")

print("\n")

if 'department' in df.columns:
    df['department_encoded'] = le.fit_transform(df['department'])
    print("Department Mappings:")
    for i, item in enumerate(le.classes_):
        print(f"{item}: {i}")
else:
    print("Department column not found in the DataFrame.")

print("\nFinal DataFrame head after label encoding:")
```

```
Job Location Mappings:
London: 0
New York: 1
Paris: 2
london: 3
new york: 4
paris: 5


Department Mappings:
hr: 0
it: 1
marketing: 2
sales: 3

Final DataFrame head after label encoding:
```

	employee_id	department	job_location	salary	years_experience	job_location_encoded	department_encoded
0	1	sales	New York	60000	2.0	1	3
1	2	marketing	London	75000	5.0	0	2
2	3	it	Paris	80000	3.0	2	1
3	4	hr	New York	55000	1.0	1	0
4	5	sales	London	62000	2.0	0	3